Optimization Methods for a Reduced PDE-Constrained Optimal Control Problem (Ex3)

Fachseminar Report

January 9, 2026

Overview

This report summarizes the mathematical formulation, discretization, and optimization methods implemented in the `Code/` folder for the Ex3 problem. The domain is $\Omega = (0,1)^2$, with homogeneous Dirichlet boundary conditions for the state and adjoint. The control acts on a circular subregion ω centered at $(\frac{1}{2}, \frac{1}{2})$ with radius r.

1 Problem Formulation

We seek a control $u \in L^2(\Omega)$ that minimizes

$$F(u) = \tfrac{1}{2}\,\|y(u) - y_d\|^2_{L^2(\Omega)} + \tfrac{\beta}{2}\,\|u\|^2_{L^2(\Omega)}, \tag{1}$$

subject to the elliptic state equation

$$-\Delta y = \chi_\omega\, u \quad \text{in } \Omega, \tag{2}$$

$$y = 0 \quad \text{on } \partial\Omega. \tag{3}$$

The desired state is

$$y_d(x) = 0.5 - \max\{\,|x - 0.5|,\, |y - 0.5|\,\}, \tag{4}$$

which creates a pyramid-like target profile. Here, $\beta > 0$ is a Tikhonov parameter and χ_ω is the indicator of the control region.

Weak Formulation. Using test functions $v \in H^1_0(\Omega)$, the weak form reads: find $y \in H^1_0(\Omega)$ such that

$$\int_\Omega \nabla y \cdot \nabla v\, dx = \int_\Omega \chi_\omega\, u\, v\, dx, \quad \forall v \in H^1_0(\Omega). \tag{5}$$

Control region and indicator. Let the control region be the closed disk

$$\omega \;=\; \{x \in \Omega : \|x - c\|_2 \le r\}, \quad c = (\tfrac{1}{2}, \tfrac{1}{2}),\; r \in (0, \tfrac{1}{2}). \tag{6}$$

Its characteristic (indicator) function $\chi_\omega : \Omega \to \{0,1\}$ is

$$\chi_\omega(x) \;=\; \begin{cases} 1, & x \in \omega, \\ 0, & x \notin \omega. \end{cases} \tag{7}$$

Consequently, for any integrable ϕ we have $\int_\Omega \chi_\omega\, \phi\, dx = \int_\omega \phi\, dx$, so the right-hand side of the weak form restricts the control action to ω.

2 Discretization

We use conforming $P1$ finite elements on a triangular mesh of Ω, denoted by $\mathcal{T}_h$. The state space $V_h \subset H_0^1(\Omega)$ and the control space $U_h \subset L^2(\Omega)$ share the same $P1$ basis (no boundary conditions applied to U_h). Let $A \in \mathbb{R}^{n\times n}$ be the stiffness matrix, $M \in \mathbb{R}^{n\times n}$ the mass matrix on V_h, $M_U \in \mathbb{R}^{n\times n}$ the L^2 mass on U_h, and $B \in \mathbb{R}^{n\times n}$ the discrete control operator for χ_ω. With Dirichlet boundary conditions applied to A only, the reduced mapping is

$$y(u) = A^{-1} B\, u. \tag{8}$$

The discrete objective becomes

$$F(u) = \tfrac{1}{2}(y - y_d)^T M (y - y_d) + \tfrac{\beta}{2}\, u^T M_U u. \tag{9}$$

The adjoint p solves

$$A\, p = M\,(y - y_d), \tag{10}$$

leading to the reduced gradient and Hessian in U_h with the M_U inner product:

$$\nabla F(u) = M_U^{-1} B^T p + \beta\, u, \tag{11}$$

$$Hv = M_U^{-1} B^T A^{-1} M A^{-1} B v + \beta\, v. \tag{12}$$

A Lipschitz constant L for the gradient can be estimated by a power iteration on H.

3 Implementation Notes

The implementation in `Code/reduced_oc_model.py` assembles A, M_U, and B using FEniCS. Dirichlet boundary conditions are enforced on A only. Sparse factorizations are reused for state/adjoint solves and for applying M_U^{-1}. The target y_d is interpolated from its expression. The model exposes

- `state(u)`, `adjoint(z)`, `cost(u)`, `grad_U(u)`, `hess_U(v)`;
- inner products `dot_U` and `norm_U`;
- `estimate_L()` for a gradient Lipschitz bound.

4 Matrix Summary

The FE discretization produces the following matrices (see the separate Discretization Note for exact code line references and details):

extbfMatrix	**Symbol**	**Definition**
Stiffness	A	$A_{ij} = \int_\Omega \nabla\varphi_i \cdot \nabla\varphi_j\, dx$ (Dirichlet BCs applied).
State Mass	M	$M_{ij} = \int_\Omega \varphi_i\, \varphi_j\, dx$.
Control Mass	M_U	$(M_U)_{ij} = \int_\Omega \varphi_i\, \varphi_j\, dx$ (inner product in U_h).
Control Coupling	B	$B_{ij} = \int_\omega \varphi_i\, \varphi_j\, dx$ (restriction by χ_ω).

5 Optimization Methods

All methods work in the control space $(U_h, \langle\cdot,\cdot\rangle_{M_U})$ with $\nabla F(u)$ as above.

Barzilai–Borwein (BB)

Given iterates (u_k, g_k) with $g_k = \nabla F(u_k)$, define $s_k = u_k - u_{k-1}$ and $d_k = g_k - g_{k-1}$. The BB steps alternate between

$$\alpha_k^{(1)} = \frac{\langle d_k, d_k \rangle_{M_U}}{\langle s_k, d_k \rangle_{M_U}}, \qquad \alpha_k^{(2)} = \frac{\langle s_k, d_k \rangle_{M_U}}{\langle s_k, s_k \rangle_{M_U}}, \tag{13}$$

with the update $u_{k+1} = u_k - \alpha_k^{-1} g_k$. This often accelerates steepest descent at negligible cost.

Fixed-Step Gradient Descent

With a Lipschitz estimate $L \geq L_F$, choose $\alpha = 1/L$ and iterate

$$u_{k+1} = u_k - \alpha \nabla F(u_k). \tag{14}$$

Convergence is guaranteed for $\alpha \in (0, 2/L_F)$, and the method is simple and robust when L is not too conservative.

Nesterov Acceleration (FISTA-style)

Initialize $t_0 = 1$, $y_0 = u_0$, and iterate for $k \geq 0$:

$$u_{k+1} = y_k - \tfrac{1}{L} \nabla F(y_k), \tag{15}$$

$$t_{k+1} = \tfrac{1+\sqrt{1+4t_k^2}}{2}, \tag{16}$$

$$y_{k+1} = u_{k+1} + \tfrac{t_k - 1}{t_{k+1}}(u_{k+1} - u_k). \tag{17}$$

A simple restart (resetting momentum when F increases) improves robustness.

6 Figures

Figures are generated by `Code/generate_report_assets.py` and saved to `results/`.

7 Conclusions

The reduced formulation enables efficient first-order optimization by reusing sparse factorizations for state/adjoint solves. BB and Nesterov typically converge faster than fixed-step GD, while GD(1/L) provides a strong baseline when a reliable Lipschitz estimate is available.

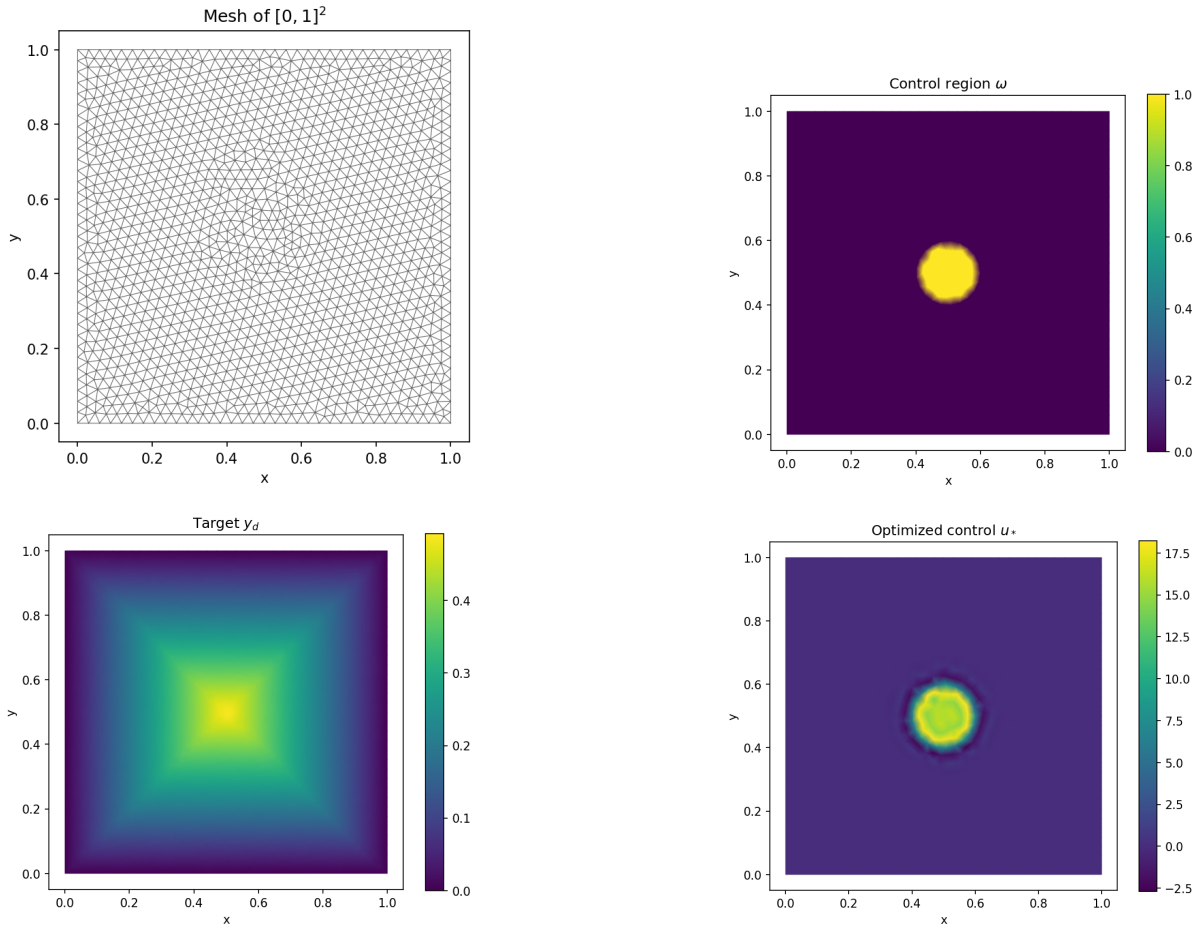

Figure 1: Top: mesh and control region ω. Bottom-left: target y_d. Bottom-right: optimized control u_*.

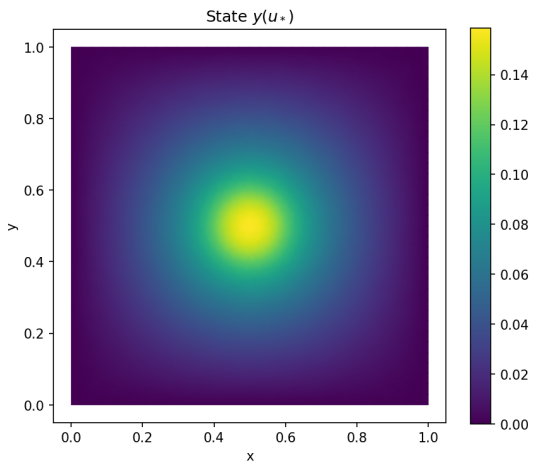

Figure 2: State $y(u_*)$ corresponding to the optimized control.

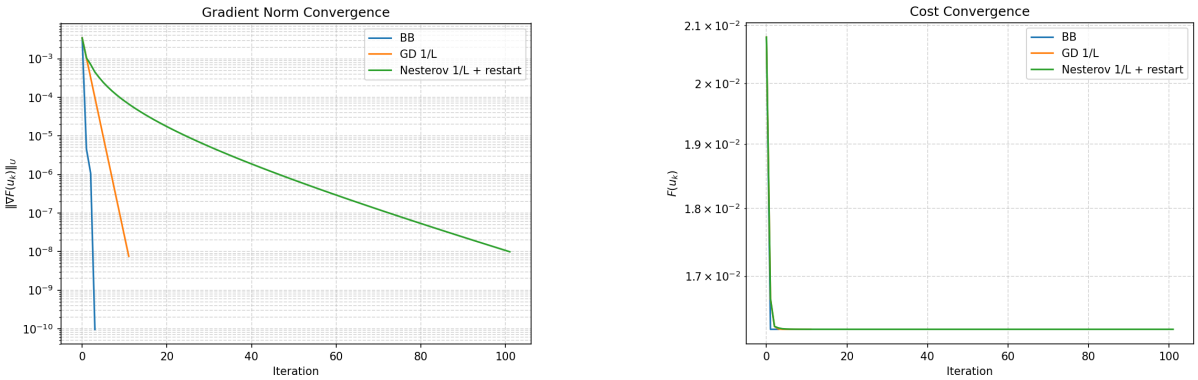

Figure 3: Convergence of gradient norm and objective for BB, GD with $1/L$, and Nesterov with restart.